

Analyzing the Rock5B VPU Driver

Finding Kernel Dependencies During AV1 Hardware Decoding

Till Bégue

August 10, 2026

ETH Zürich

Motivation

Device Drivers



privileged code translating generic OS calls → device-specific hardware operations

Device Drivers



privileged code translating generic OS calls → device-specific hardware operations

- **Hardware Abstraction:** Hides vendor-specific complexity behind an interface.

Device Drivers



privileged code translating generic OS calls → device-specific hardware operations

- **Hardware Abstraction:** Hides vendor-specific complexity behind an interface.
- **Request Translation:** Converts user-space system calls (e.g., `ioctl`) into hardware commands.

Device Drivers



privileged code translating generic OS calls → device-specific hardware operations

- **Hardware Abstraction:** Hides vendor-specific complexity behind an interface.
- **Request Translation:** Converts user-space system calls (e.g., `ioctl`) into hardware commands.
- **Polymorphic Dispatch:** Uses function-pointers (ops structs) to route execution.

Device Drivers



privileged code translating generic OS calls → device-specific hardware operations

- **Hardware Abstraction:** Hides vendor-specific complexity behind an interface.
- **Request Translation:** Converts user-space system calls (e.g., `ioctl`) into hardware commands.
- **Polymorphic Dispatch:** Uses function-pointers (ops structs) to route execution.
- **Monolithic Execution:** Shares the kernel's address space

Attack Surface of Drivers

The Core Issue

Executing large, complex, vendor-specific, and under-audited driver codebases with privileged access.

Attack Surface of Drivers

The Core Issue

Executing large, complex, vendor-specific, and under-audited driver codebases with privileged access.

Evidence of Systemic Risk:

- **Massive Reachability** (Maar et al., '25): 131 devices analyzed → 11 reachable drivers, 50 n-day vulnerabilities.

Attack Surface of Drivers

The Core Issue

Executing large, complex, vendor-specific, and under-audited driver codebases with privileged access.

Evidence of Systemic Risk:

- **Massive Reachability** (Maar et al., '25): 131 devices analyzed → 11 reachable drivers, 50 n-day vulnerabilities.
- **Active Exploitation** (Project Zero, '26): Pixel 0-click exploit chain in the wild (codec → driver → kernel root).

Attack Surface of Drivers

The Core Issue

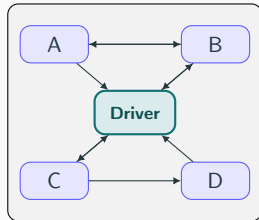
Executing large, complex, vendor-specific, and under-audited driver codebases with privileged access.

Evidence of Systemic Risk:

- **Massive Reachability** (Maar et al., '25): 131 devices analyzed → 11 reachable drivers, 50 n-day vulnerabilities.
- **Active Exploitation** (Project Zero, '26): Pixel 0-click exploit chain in the wild (codec → driver → kernel root).
- **Domain-Specific Flaws** (Vasquez et al., '23): Severe structural vulnerabilities inherent to stateless video decoders.

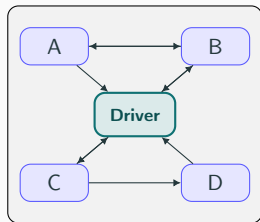
Related Work

Monolithic

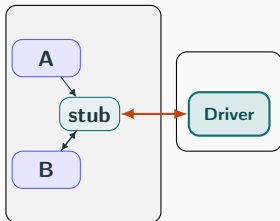


Related Work

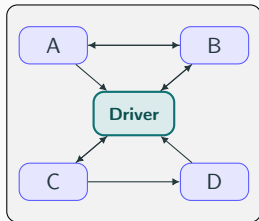
Monolithic



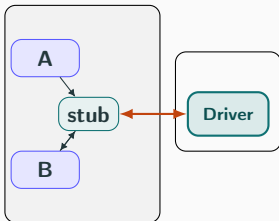
Isolated



Monolithic



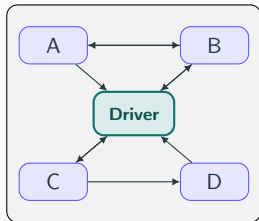
Isolated



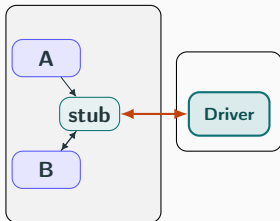
Prior Approaches & Limitations:

- **Nooks** (Swift et al. '02)
Wrapper-based isolation.
Gap: Same address space

Monolithic



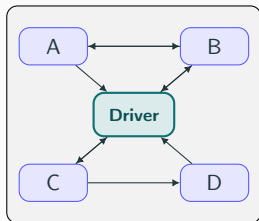
Isolated



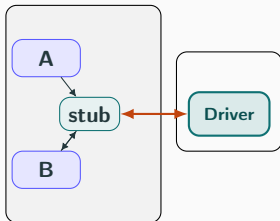
Prior Approaches & Limitations:

- **Nooks** (Swift et al. '02)
Wrapper-based isolation.
Gap: Same address space
- **LXD**s (Narayanan et al '19)
Hardware-isolated execution domains.
Gap: Requires fully manual boundary mapping.

Monolithic



Isolated



Prior Approaches & Limitations:

- **Nooks** (Swift et al. '02)
Wrapper-based isolation.
Gap: Same address space
- **LXD**s (Narayanan et al '19)
Hardware-isolated execution domains.
Gap: Requires fully manual boundary mapping.
- **kSplit** (Huang et al. '22)
Automated LVD interface generation.
Gap: Fails on complex, high-bandwidth subsystems.

Why Extraction is Non-Trivial

The Missing Link

We must map the exact “life-support” dependencies a driver needs to survive outside the monolithic kernel.

Why Extraction is Non-Trivial

The Missing Link

We must map the exact “life-support” dependencies a driver needs to survive outside the monolithic kernel.

Roadblocks to Automated Extraction:

- Deeply entangled with kernel frameworks (V4L2, DMA-buf).

Why Extraction is Non-Trivial

The Missing Link

We must map the exact “life-support” dependencies a driver needs to survive outside the monolithic kernel.

Roadblocks to Automated Extraction:

- Deeply entangled with kernel frameworks (V4L2, DMA-buf).
- Real-time video relies on complex, zero-copy DMA buffer sharing.

Why Extraction is Non-Trivial

The Missing Link

We must map the exact “life-support” dependencies a driver needs to survive outside the monolithic kernel.

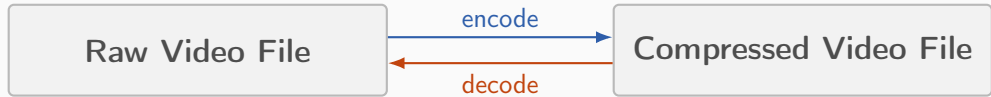
Roadblocks to Automated Extraction:

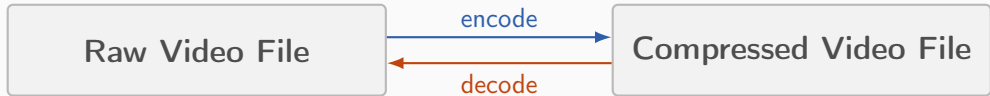
- Deeply entangled with kernel frameworks (V4L2, DMA-buf).
- Real-time video relies on complex, zero-copy DMA buffer sharing.
- Hardware state driven entirely via intricate `ioctl`s and indirect pointers.

What is the minimal set of kernel objects the Hantro driver needs to run a AV1 decoding process isolated?

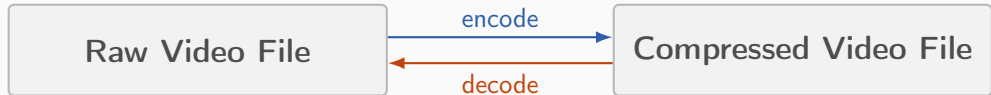
Background

AV1 Video Codec

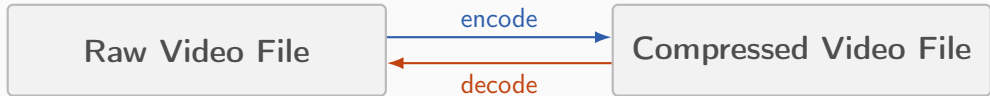




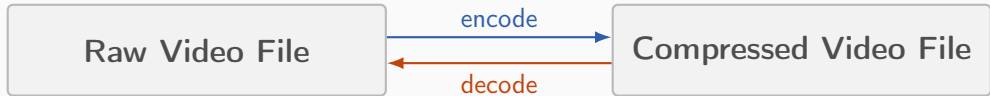
- Royalty-free codec, ~30% better compression than HEVC (h.265)



- Royalty-free codec, ~30% better compression than HEVC (h.265)
- Compression cost $\Rightarrow$ hardware decode required

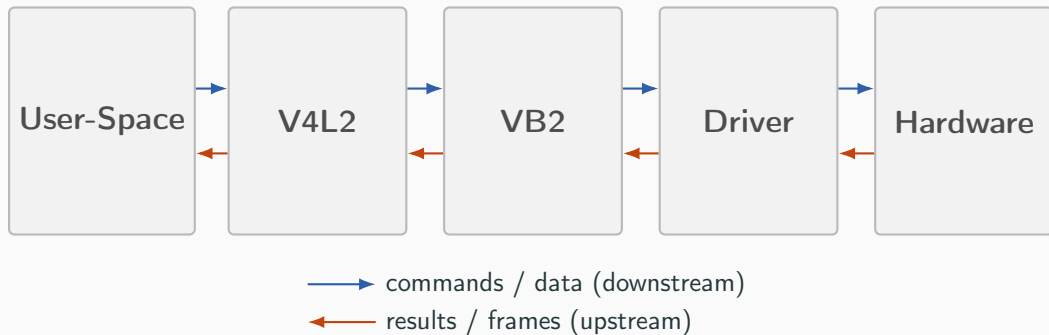


- Royalty-free codec, ~30% better compression than HEVC (h.265)
- Compression cost $\Rightarrow$ hardware decode required
- **Stateless** decoder: user-space parses bitstream, hardware only executes



- Royalty-free codec, ~30% better compression than HEVC (h.265)
- Compression cost $\Rightarrow$ hardware decode required
- **Stateless** decoder: user-space parses bitstream, hardware only executes
- Media Request API bundles buffer + per-frame controls atomically

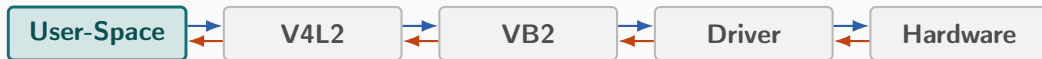
Linux Video Subsystem



User-Space

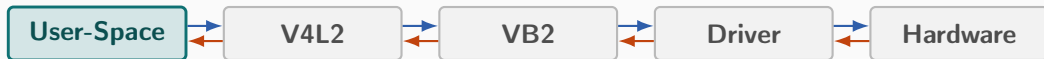


User-Space



Role

- User-space element opens `/dev/videoN`
- Owns the buffer lifecycle from the application side
- No kernel-internal state; purely a syscall/ioctl client

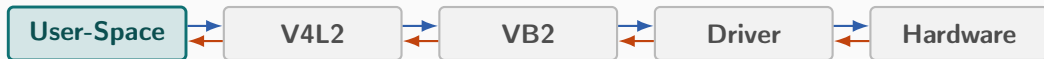


Role

- User-space element opens `/dev/videoN`
- Owns the buffer lifecycle from the application side
- No kernel-internal state; purely a syscall/ioctl client

Receives

- Decoded frames, Format / capability answers, Stream-ready events



Role

- User-space element opens `/dev/videoN`
- Owns the buffer lifecycle from the application side
- No kernel-internal state; purely a syscall/ioctl client

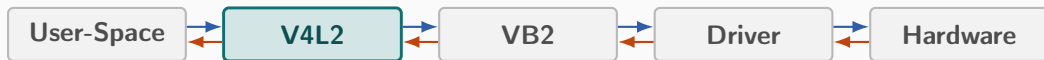
Receives

- Decoded frames, Format / capability answers, Stream-ready events

Provides

- ioctl commands, Compressed bitstream, Control values

V4L2 Core



V4L2 Core



Role

- ioctl handling
- mem2mem (M2M): pairs OUTPUT/CAPTURE



Role

- ioctl handling
- mem2mem (M2M): pairs OUTPUT/CAPTURE

Receives

- ioctl commands and OUTPUT (compressed) buffers from user-space
- Completed CAPTURE buffers and capability answers from VB2/M2M

V4L2 Core



Role

- ioctl handling
- mem2mem (M2M): pairs OUTPUT/CAPTURE

Receives

- ioctl commands and OUTPUT (compressed) buffers from user-space
- Completed CAPTURE buffers and capability answers from VB2/M2M

Provides

- Dispatches ioctls, Delegates buffer operations to VB2 using M2M
- Returns decoded frames and format/cap info back to user-space





Role

- State machine: DEQUEUED → QUEUED → ACTIVE → DONE
- DMA/CMA/vmalloc-backed memory



Role

- State machine: DEQUEUED → QUEUED → ACTIVE → DONE
- DMA/CMA/vmalloc-backed memory

Receives

- Buffer requests & stream control from V4L2/M2M
- Driver Buffer-done notifications



Role

- State machine: DEQUEUED → QUEUED → ACTIVE → DONE
- DMA/CMA/vmalloc-backed memory

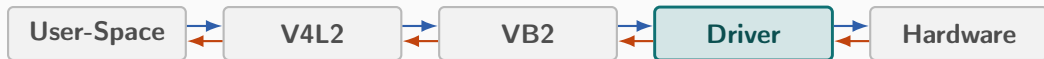
Receives

- Buffer requests & stream control from V4L2/M2M
- Driver Buffer-done notifications

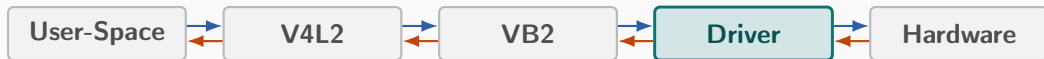
Provides

- Invokes driver callbacks
- Returns DONE buffers to V4L2/M2M

Codec Driver



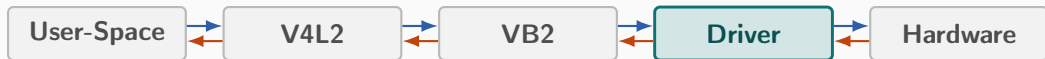
Codec Driver



Role

- Translates incoming data into hardware registers, stateless
- Provides Hardware specific information

Codec Driver



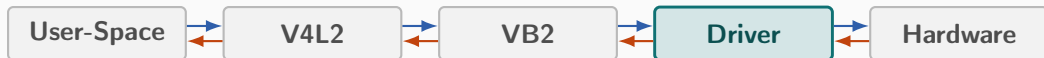
Role

- Translates incoming data into hardware registers, stateless
- Provides Hardware specific information

Receives

- Buffers, controls & callbacks from VB2
- Decode-complete IRQ from hardware

Codec Driver



Role

- Translates incoming data into hardware registers, stateless
- Provides Hardware specific information

Receives

- Buffers, controls & callbacks from VB2
- Decode-complete IRQ from hardware

Provides

- Register programming
- Marks buffer DONE; answers capability queries





Role

- Performs decoding process on hardware



Role

- Performs decoding process on hardware

Receives

- Compressed bitstream & reference frames (DMA-mapped)
- Job parameters from the driver



Role

- Performs decoding process on hardware

Receives

- Compressed bitstream & reference frames (DMA-mapped)
- Job parameters from the driver

Provides

- Decoded frame in DMA output buffer
- Completion IRQ

Kernel Polymorphism

```
struct vb2_ops {  
    int  (*buf_prepare)(struct vb2_buffer *vb);  
    void (*buf_queue)  (struct vb2_buffer *vb);  
};
```

```
static const struct vb2_ops hantro_qops = {  
    .buf_prepare = hantro_buf_prepare,  
    .buf_queue   = hantro_buf_queue,  
};
```

- ops->buf_prepare(vb) — concrete target unknown at the call site
- Binding happens at **runtime**, during driver registration
- grep / go-to-definition: cannot resolve

Methodology

Environment Setup

- Radxa Rock5B using rk3588
- OpenCCA 6.12 Linux kernel
- GStreamer 1.24
- Config Adjustments

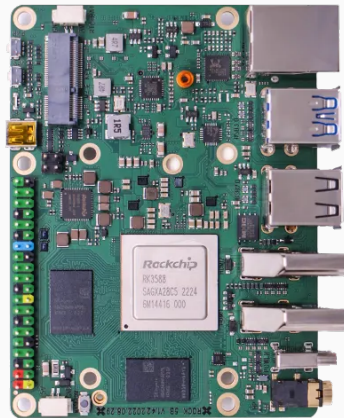
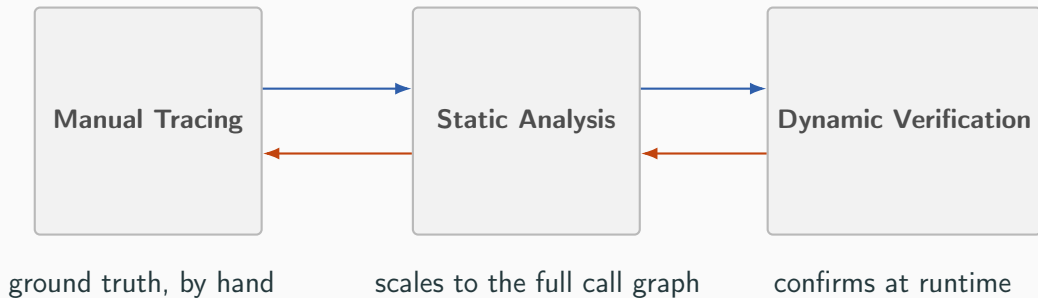


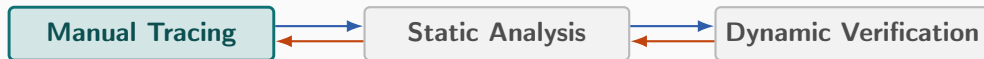
Figure 1: Radxa Rock5B

Three-Stage Pipeline



Manual Tracing

- Determine Entry Points
- Manually tracing REGBUFS
- Provides ground truth and intuition



CodeQL

- Compiles source code into a queryable relational database.

CodeQL

- Compiles source code into a queryable relational database.
- logical predicates over the AST, tracing data flows.

CodeQL

- Compiles source code into a queryable relational database.
- logical predicates over the AST, tracing data flows.
- Custom queries via `exists()`



Using a custom CodeQL Library following queries were built:

- Call Graph
- Ops Struct implementation
- Syntactic Struct Access
- Taint Tracking Struct Access

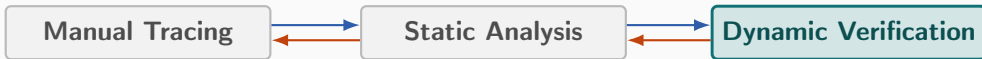


Static Analysis & Ops Structs

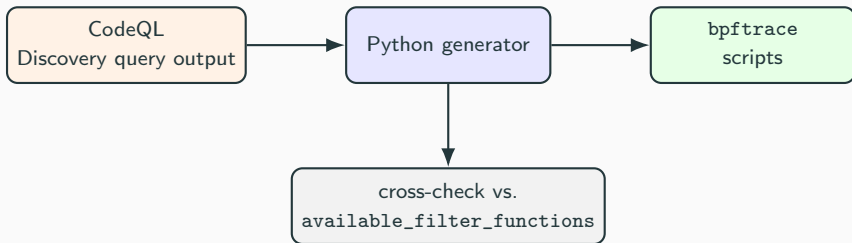
```
predicate calls(Function caller, Function callee) {  
  // Case 1: Direct function calls  
  exists(Call c | c.getEnclosingFunction() = caller and callee = c.getTarget())  
  or  
  // Case 2: Indirect calls  
  exists(ExprCall c, Field f | c.getEnclosingFunction() = caller and  
    resolvesOpsField(f, callee) and  
    isActiveImplementation(callee))  
  or  
  // Case 3: Function pointers passed as callback arguments  
  exists(FunctionCall c, FunctionAccess fa |  
    c.getEnclosingFunction() = caller and  
    fa = c.getArgument(_) and callee = fa.getTarget()))}
```

Dynamic Tracing & the Inlining Problem

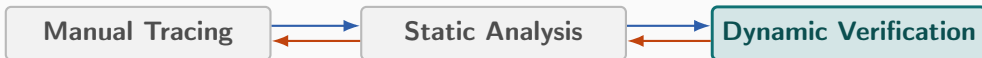
- bpftrace: high-level syntax over eBPF
- kprobes
- Inlined functions have **no addressable entry point**



Automated Trace Workflow



- Traceable → invocation-counting kprobe
- Untraceable → flagged as comments



Results

Call Graph

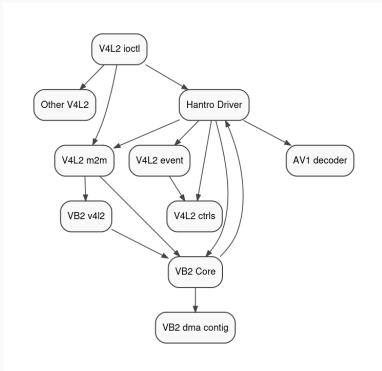


Figure 2: File-level graph of REGBUFS

Call Graph



Figure 2: File-level graph of REQBUFS

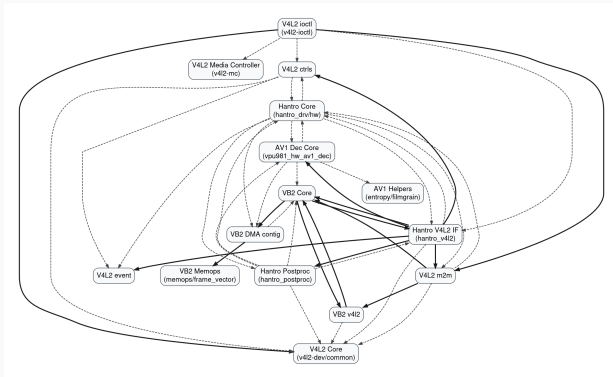


Figure 3: Full file-level graph

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)
- 34 kernel structs accessed across the boundary
 - Syntactic analysis sufficed—taint tracking yielded zero true positives.

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)
- 34 kernel structs accessed across the boundary
 - Syntactic analysis sufficed—taint tracking yielded zero true positives.

Unused paths: 44 of 208 probed Functions

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)
- 34 kernel structs accessed across the boundary
 - Syntactic analysis sufficed—taint tracking yielded zero true positives.

Unused paths: 44 of 208 probed Functions

- unused VB2 buffer allocation

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)
- 34 kernel structs accessed across the boundary
 - Syntactic analysis sufficed—taint tracking yielded zero true positives.

Unused paths: 44 of 208 probed Functions

- unused VB2 buffer allocation
- Post-processor & Filmgrain

Key Findings

Found Dependencies:

- 11 driver-implemented `ioctl`s → entry points
- 10 ops struct implementations resolved & confirmed
- 13 cross-boundary dependency edges (DMA, MMIO, VB2, M2M, ...)
- 34 kernel structs accessed across the boundary
 - Syntactic analysis sufficed—taint tracking yielded zero true positives.

Unused paths: 44 of 208 probed Functions

- unused VB2 buffer allocation
- Post-processor & Filmgrain
- Debug, Recovery, Control paths

Conclusion & Future Work

- Three-stage methodology: manual $\rightarrow$ static $\rightarrow$ dynamic

- Three-stage methodology: manual $\rightarrow$ static $\rightarrow$ dynamic
- Precisely mapped `ioctls`, `ops structs`, and kernel dependencies for the Hantro AV1 driver

- Three-stage methodology: manual → static → dynamic
- Precisely mapped `ioctls`, `ops` structs, and kernel dependencies for the Hantro AV1 driver
- Reusable: CodeQL library & bpftrace generator apply beyond this driver

- **Driver extraction:** stub the shared structs, run the Hantro driver outside the kernel

- **Driver extraction:** stub the shared structs, run the Hantro driver outside the kernel
- **Extend Dynamic Trace:** validate diverse workloads, struct access, resolve inlined call sites

Thank you.

Questions?